E-Commerce Product Analytics: Customer Engagement, Conversion & Retention

An End-to-End Data Analytics Project Using Python, SQL & Power BI

📌 About the Project

This project focuses on analyzing customer behavior across the e-commerce lifecycle to understand how users interact with the platform, discover products, make purchases, and return over time.

Using five interconnected datasets covering 10,000 customers, 1,000 products, 120,000 transactions, 80,000 browsing sessions, and 25,000 product reviews, the project explores customer engagement, purchasing patterns, conversion performance, and retention.

Python will be used for data cleaning, exploratory data analysis, feature engineering, customer segmentation, and predictive modeling. SQL will support relational data analysis and the calculation of key product metrics, while Power BI will bring the findings together in interactive dashboards.

The analysis will cover Daily, Weekly, and Monthly Active Users (DAU, WAU, MAU), funnel analysis, cohort-based retention, RFM segmentation, customer lifetime value prediction, churn prediction, conversion-rate optimization, and product recommendation through market-basket analysis.

The ultimate objective is to transform raw e-commerce data into actionable business insights that help identify opportunities to improve customer engagement, increase conversions, strengthen retention, and support data-driven product decisions.

Tools & Technologies: Python, Pandas, NumPy, Matplotlib, Seaborn, SQL, PostgreSQL, Power BI, Scikit-learn.

Dataset: Synthetic e-commerce data designed for analytical learning, benchmarking, and prototyping. Findings represent patterns within the generated data and should not be interpreted as evidence of real-world customer behavior.

In [5]:
```
### Importing all libraries

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from scipy import stats
import statsmodels.api as sm
from sklearn.model_selection import (
    train_test_split,
    cross_val_score,
    cross_validate,
    StratifiedKFold,
    KFold,
    GridSearchCV
)

from sklearn.preprocessing import (
    StandardScaler,
```

```
    MinMaxScaler,
    OneHotEncoder,
    LabelEncoder
)

from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer
from sklearn.linear_model import LinearRegression, Ridge, Lasso
from sklearn.ensemble import (
    RandomForestRegressor,
    GradientBoostingRegressor,
    RandomForestClassifier,
    GradientBoostingClassifier,
    HistGradientBoostingRegressor,
    HistGradientBoostingClassifier
)

from sklearn.metrics import (
    mean_absolute_error,
    mean_squared_error,
    r2_score,
    roc_auc_score,
    classification_report,
    confusion_matrix,
    precision_score,
    recall_score,
    f1_score,
    accuracy_score,
    precision_recall_curve,
    average_precision_score
)
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score
from mlxtend.frequent_patterns import (
    apriori,
    fpgrowth,
    association_rules
)
from sqlalchemy import create_engine, text
from pathlib import Path
from datetime import datetime
import warnings
pd.set_option("display.max_columns", None)
pd.set_option("display.max_rows", 100)
pd.set_option("display.float_format", "{:,.2f}".format)
sns.set_theme(style="whitegrid")
plt.rcParams["figure.figsize"] = (12, 6)
warnings.filterwarnings("default")

print("All project libraries imported successfully!")
```

```
All project libraries imported successfully!
```

In [7]:

```
# Folder containing your five CSV files
DATA_PATH = Path(r"C:\Users\papus\OneDrive\Documents\Project File")

# Load datasets
customers = pd.read_csv(DATA_PATH / "customers.csv")
products = pd.read_csv(DATA_PATH / "products.csv")
transactions = pd.read_csv(DATA_PATH / "transactions.csv")
sessions = pd.read_csv(DATA_PATH / "sessions.csv")
reviews = pd.read_csv(DATA_PATH / "reviews.csv")
```

```
print("All five datasets loaded successfully!")
```

```
All five datasets loaded successfully!
```

In [8]:
```
datasets = {
    "Customers": customers,
    "Products": products,
    "Transactions": transactions,
    "Sessions": sessions,
    "Reviews": reviews
}

for name, df in datasets.items():
    print(f"{name}: {df.shape[0]:,} rows × {df.shape[1]} columns")
```

```
Customers: 10,000 rows × 10 columns
Products: 1,000 rows × 11 columns
Transactions: 120,000 rows × 11 columns
Sessions: 80,000 rows × 10 columns
Reviews: 25,000 rows × 8 columns
```

In [9]:
```
for name, df in datasets.items():
    print(f"\n{'=' * 50}")
    print(f"DATASET: {name}")
    print(f"{'=' * 50}")

    print("\nColumns and data types:")
    print(df.dtypes)

    print("\nFirst 5 rows:")
    display(df.head())
```

```
==================================================
DATASET: Customers
==================================================

Columns and data types:
customer_id        object
signup_date        object
age                 int64
gender             object
country            object
segment            object
is_churned          int64
lifetime_value    float64
email_opt_in        int64
has_app             int64
dtype: object

First 5 rows:
```

	customer_id	signup_date	age	gender	country	segment	is_churned	lifetime_value	email_opt_in
0	C00000	13-02-2020	28	M	BR	Premium	0	1,595.27	0
1	C00001	01-10-2021	22	Prefer not to say	FR	Regular	0	1,160.61	1
2	C00002	26-06-2022	30	F	US	VIP	0	3,093.32	1
3	C00003	21-12-2021	48	F	US	Premium	1	2,131.08	1
4	C00004	16-09-2021	37	M	CA	Budget Shopper	0	583.39	0

```
==================================================
DATASET: Products
==================================================

Columns and data types:
product_id         object
product_name       object
category           object
brand              object
price             float64
avg_rating        float64
num_ratings         int64
stock_quantity      int64
discount_pct        int64
is_featured         int64
weight_kg         float64
dtype: object

First 5 rows:
```

	product_id	product_name	category	brand	price	avg_rating	num_ratings	stock_quantity	dis
0	P0000	PetLife Health #0	Health	PetLife	34.82	4.00	137	126	
1	P0001	GlowUp Beauty #1	Beauty	GlowUp	53.35	5.00	494	51	
2	P0002	EcoLiving Office Supplies #2	Office Supplies	EcoLiving	51.45	3.50	433	928	
3	P0003	DigiTools Electronics #3	Electronics	DigiTools	179.90	4.00	267	127	
4	P0004	StyleMax Pet Supplies #4	Pet Supplies	StyleMax	12.54	3.70	402	580	

```
==================================================
DATASET: Transactions
==================================================

Columns and data types:
transaction_id       object
customer_id          object
product_id           object
transaction_date     object
quantity              int64
unit_price          float64
total_amount        float64
discount_applied      int64
status               object
payment_method       object
shipping_cost       float64
dtype: object

First 5 rows:
```

	transaction_id	customer_id	product_id	transaction_date	quantity	unit_price	total_amount	discoun
0	T077767	C08119	P0932	01-01-2023 00:23	1	23.87	23.87	
1	T021376	C08883	P0889	01-01-2023 00:26	1	31.53	31.53	
2	T117783	C05855	P0065	01-01-2023 00:45	1	67.75	67.75	
3	T035185	C09437	P0200	01-01-2023 01:04	1	151.06	151.06	
4	T092643	C00760	P0881	01-01-2023 01:06	1	11.97	11.97	



```
==================================================
DATASET: Sessions
==================================================

Columns and data types:
session_id          object
customer_id         object
session_date        object
device              object
channel             object
duration_seconds     int64
pages_viewed         int64
converted            int64
bounced              int64
cart_additions       int64
dtype: object

First 5 rows:
```

	session_id	customer_id	session_date	device	channel	duration_seconds	pages_viewed	converted
0	S049064	C03202	01-01-2023 00:24	desktop	direct	72	1	1
1	S015939	C09526	01-01-2023 00:42	mobile	organic	36	1	0
2	S003166	C07034	01-01-2023 00:55	desktop	social	284	5	0
3	S039733	C05327	01-01-2023 01:33	mobile	direct	805	10	0
4	S045293	C05338	01-01-2023 01:35	tablet	direct	207	3	0

```
==================================================
DATASET: Reviews
==================================================

Columns and data types:
review_id            object
customer_id          object
product_id           object
review_date          object
rating                int64
review_text          object
helpful_votes         int64
verified_purchase     int64
dtype: object

First 5 rows:
```

	review_id	customer_id	product_id	review_date	rating	review_text	helpful_votes	verified_purchase
0	R22520	C06622	P0077	01-01-2023	4	Great product! Highly recommend.	0	0
1	R11296	C01152	P0726	01-01-2023	5	Love it! Exactly what I needed.	3	1
2	R15342	C05432	P0167	01-01-2023	3	Average quality, meets expectations.	42	1
3	R06757	C00693	P0443	01-01-2023	3	Not bad, not great.	43	1
4	R08506	C05569	P0972	01-01-2023	4	Amazing value for the price.	46	1

In [10]:
```python
# Identifying potential date/time columns in all five datasets

for name, df in datasets.items():
    print(f"\n{'=' * 50}")
    print(f"{name}")
```

```
    print('=' * 50)

    date_columns = [
        col for col in df.columns
        if any(keyword in col.lower()
               for keyword in ["date", "time", "timestamp"])
    ]

    print("Potential date/time columns:", date_columns)

    if date_columns:
        display(df[date_columns].head())
        print("\nCurrent data types:")
        print(df[date_columns].dtypes)
```

```
==================================================
Customers
==================================================
Potential date/time columns: ['signup_date', 'lifetime_value']
```

	signup_date	lifetime_value
0	13-02-2020	1,595.27
1	01-10-2021	1,160.61
2	26-06-2022	3,093.32
3	21-12-2021	2,131.08
4	16-09-2021	583.39

```
Current data types:
signup_date        object
lifetime_value    float64
dtype: object

==================================================
Products
==================================================
Potential date/time columns: []

==================================================
Transactions
==================================================
Potential date/time columns: ['transaction_date']
```

	transaction_date
0	01-01-2023 00:23
1	01-01-2023 00:26
2	01-01-2023 00:45
3	01-01-2023 01:04
4	01-01-2023 01:06

```
Current data types:
transaction_date    object
dtype: object

==================================================
Sessions
==================================================
Potential date/time columns: ['session_date']
```

	session_date
0	01-01-2023 00:24
1	01-01-2023 00:42
2	01-01-2023 00:55
3	01-01-2023 01:33
4	01-01-2023 01:35

```
Current data types:
session_date    object
dtype: object

==================================================
Reviews
==================================================
Potential date/time columns: ['review_date']
```

	review_date
0	01-01-2023
1	01-01-2023
2	01-01-2023
3	01-01-2023
4	01-01-2023

```
Current data types:
review_date    object
dtype: object
```

In [11]:

```python
# Converting confirmed date columns to datetime

date_columns = {
    "customers": ["signup_date"],
    "products": [],
    "transactions": ["transaction_date"],
    "sessions": ["session_date"],
    "reviews": ["review_date"]
}

dataframes = {
    "customers": customers,
    "products": products,
    "transactions": transactions,
    "sessions": sessions,
    "reviews": reviews
}

for name, columns in date_columns.items():
    df = dataframes[name]

    for col in columns:
        if col in df.columns:
            df[col] = pd.to_datetime(df[col], errors="coerce")

            print(f"{name}.{col}: {df[col].dtype}")
```

```
C:\Users\papus\AppData\Local\Temp\ipykernel_9948\11101023.py:24: UserWarning: Parsing dates in %d-%m-%Y format when dayfirst=False (the default) was specified. Pass `dayfirst=True` or specify a format to silence this warning.
  df[col] = pd.to_datetime(df[col], errors="coerce")
customers.signup_date: datetime64[ns]
transactions.transaction_date: datetime64[ns]
sessions.session_date: datetime64[ns]
reviews.review_date: datetime64[ns]
```

In [12]:
```python
for name, df in datasets.items():
    print(f"\n{name}")

    for col in df.columns:
        if pd.api.types.is_datetime64_any_dtype(df[col]):
            print(f"{col}: {df[col].dtype}")
```

```
Customers
signup_date: datetime64[ns]

Products

Transactions
transaction_date: datetime64[ns]

Sessions
session_date: datetime64[ns]

Reviews
review_date: datetime64[ns]
```

In [13]:
```python
# Missing Value Analysis

for name, df in datasets.items():
    print(f"\n{'=' * 60}")
    print(f"Missing Values: {name}")
    print('=' * 60)

    missing = df.isnull().sum()

    missing_report = pd.DataFrame({
        "Missing_Count": missing,
        "Missing_Percentage": (missing / len(df) * 100).round(2)
    })

    display(missing_report[missing_report["Missing_Count"] > 0])
```

```
============================================================
Missing Values: Customers
============================================================
```

	Missing_Count	Missing_Percentage

```
============================================================
Missing Values: Products
============================================================
```

	Missing_Count	Missing_Percentage

```
============================================================
Missing Values: Transactions
============================================================
```

	Missing_Count	Missing_Percentage
transaction_date	72506	60.42

```
============================================================
Missing Values: Sessions
============================================================
```

	Missing_Count	Missing_Percentage
session_date	48498	60.62

```
============================================================
Missing Values: Reviews
============================================================
```

	Missing_Count	Missing_Percentage
review_date	15174	60.70

In [14]:
```python
date_checks = {
    "Transactions": (transactions, "transaction_date"),
    "Sessions": (sessions, "session_date"),
    "Reviews": (reviews, "review_date")
}

for name, (df, col) in date_checks.items():

    print(f"\n{'=' * 60}")
    print(f"{name} — {col}")
    print("=" * 60)

    print("Data type:", df[col].dtype)
    print("Missing values:", df[col].isna().sum())
    print("Missing percentage:",
          round(df[col].isna().mean() * 100, 2), "%")

    print("\nSample non-missing values:")
    print(df.loc[df[col].notna(), col].head(10).to_list())

    print("\nSample missing records:")
    display(df.loc[df[col].isna()].head(5))
```

```
============================================================
Transactions — transaction_date
============================================================
Data type: datetime64[ns]
Missing values: 72506
Missing percentage: 60.42 %

Sample non-missing values:
[Timestamp('2023-01-01 00:23:00'), Timestamp('2023-01-01 00:26:00'), Timestamp('2023-01-01 00:
45:00'), Timestamp('2023-01-01 01:04:00'), Timestamp('2023-01-01 01:06:00'), Timestamp('2023-0
1-01 01:13:00'), Timestamp('2023-01-01 01:17:00'), Timestamp('2023-01-01 01:18:00'), Timestamp
('2023-01-01 01:26:00'), Timestamp('2023-01-01 01:38:00')]

Sample missing records:
```

	transaction_id	customer_id	product_id	transaction_date	quantity	unit_price	total_amount	disc
1899	T086625	C02905	P0764	NaT	1	12.94	12.94	
1900	T087685	C09263	P0858	NaT	1	29.97	29.97	
1901	T079336	C02493	P0248	NaT	2	34.52	69.04	
1902	T087840	C06395	P0044	NaT	1	24.50	24.50	
1903	T005758	C09133	P0195	NaT	1	32.87	32.87	

```
============================================================
Sessions — session_date
============================================================
Data type: datetime64[ns]
Missing values: 48498
Missing percentage: 60.62 %

Sample non-missing values:
[Timestamp('2023-01-01 00:24:00'), Timestamp('2023-01-01 00:42:00'), Timestamp('2023-01-01 00:
55:00'), Timestamp('2023-01-01 01:33:00'), Timestamp('2023-01-01 01:35:00'), Timestamp('2023-0
1-01 01:55:00'), Timestamp('2023-01-01 02:01:00'), Timestamp('2023-01-01 02:14:00'), Timestamp
('2023-01-01 02:14:00'), Timestamp('2023-01-01 02:17:00')]

Sample missing records:
```

	session_id	customer_id	session_date	device	channel	duration_seconds	pages_viewed	converte
1282	S037271	C01060	NaT	mobile	organic	33	1	
1283	S010894	C04994	NaT	mobile	direct	91	1	
1284	S019500	C05891	NaT	mobile	email	233	5	
1285	S014981	C03003	NaT	mobile	social	16	1	
1286	S078634	C08154	NaT	mobile	organic	507	16	

```
============================================================
Reviews — review_date
============================================================
Data type: datetime64[ns]
Missing values: 15174
Missing percentage: 60.7 %

Sample non-missing values:
[Timestamp('2023-01-01 00:00:00'), Timestamp('2023-01-01 00:00:00'), Timestamp('2023-01-01 00:
00:00'), Timestamp('2023-01-01 00:00:00'), Timestamp('2023-01-01 00:00:00'), Timestamp('2023-0
1-01 00:00:00'), Timestamp('2023-01-01 00:00:00'), Timestamp('2023-01-01 00:00:00'), Timestamp
('2023-01-01 00:00:00'), Timestamp('2023-01-01 00:00:00')]

Sample missing records:
```

	review_id	customer_id	product_id	review_date	rating	review_text	helpful_votes	verified_purcha
391	R11940	C02308	P0314	NaT	4	Excellent quality, fast shipping.	38	
392	R21376	C02492	P0014	NaT	3	Not bad, not great.	30	
393	R24624	C00815	P0829	NaT	4	Amazing value for the price.	15	
394	R04041	C09003	P0499	NaT	4	Excellent quality, fast shipping.	12	
395	R07513	C03156	P0506	NaT	4	Excellent quality, fast shipping.	10	

In [15]:
```
# Checking where missing dates begin and end

date_checks = {
    "Transactions": (transactions, "transaction_date"),
    "Sessions": (sessions, "session_date"),
    "Reviews": (reviews, "review_date")
}

for name, (df, col) in date_checks.items():
    missing_mask = df[col].isna()

    print(f"\n{'=' * 55}")
    print(name)
    print('=' * 55)

    print("First missing row position:",
          missing_mask.to_numpy().argmax()
          if missing_mask.any() else "None")

    print("Missing rows form one continuous block:",
          missing_mask.sum() > 0
          and missing_mask.iloc[
              missing_mask.to_numpy().argmax():
          ].all())
```

```
============================================================
Transactions
============================================================
First missing row position: 1899
Missing rows form one continuous block: False

============================================================
Sessions
============================================================
First missing row position: 1282
Missing rows form one continuous block: False

============================================================
Reviews
============================================================
First missing row position: 391
Missing rows form one continuous block: False
```

In [18]:
```python
from pathlib import Path
import pandas as pd

DATA_PATH = Path(r"C:\Users\papus\OneDrive\Documents\Project File")

files = {
    "Transactions": ("transactions.csv", "transaction_date"),
    "Sessions": ("sessions.csv", "session_date"),
    "Reviews": ("reviews.csv", "review_date")
}

for name, (filename, col) in files.items():
    df_original = pd.read_csv(DATA_PATH / filename)

    print(f"\n{name}")
    print("Missing dates:", df_original[col].isna().sum())
    print("Missing percentage:",
          round(df_original[col].isna().mean() * 100, 2), "%")
```

```
Transactions
Missing dates: 0
Missing percentage: 0.0 %

Sessions
Missing dates: 0
Missing percentage: 0.0 %

Reviews
Missing dates: 0
Missing percentage: 0.0 %
```

In [19]:
```python
import pandas as pd
from pathlib import Path

DATA_PATH = Path(r"C:\Users\papus\OneDrive\Documents\Project File")

transactions_raw = pd.read_csv(DATA_PATH / "transactions.csv")
sessions_raw = pd.read_csv(DATA_PATH / "sessions.csv")
reviews_raw = pd.read_csv(DATA_PATH / "reviews.csv")

print("Original files reloaded successfully!")
```

```
Original files reloaded successfully!
```

In [20]:
```python
comparisons = {
    "Transactions": (
```

```
        transactions_raw["transaction_date"],
        transactions["transaction_date"]
    ),
    "Sessions": (
        sessions_raw["session_date"],
        sessions["session_date"]
    ),
    "Reviews": (
        reviews_raw["review_date"],
        reviews["review_date"]
    )
}

for name, (original, current) in comparisons.items():

    print(f"\n{'=' * 50}")
    print(name)
    print("=" * 50)

    print("Original missing dates:", original.isna().sum())
    print("Current missing dates:", current.isna().sum())

    print("Original data type:", original.dtype)
    print("Current data type:", current.dtype)
```

```
==================================================
Transactions
==================================================
Original missing dates: 0
Current missing dates: 72506
Original data type: object
Current data type: datetime64[ns]

==================================================
Sessions
==================================================
Original missing dates: 0
Current missing dates: 48498
Original data type: object
Current data type: datetime64[ns]

==================================================
Reviews
==================================================
Original missing dates: 0
Current missing dates: 15174
Original data type: object
Current data type: datetime64[ns]
```

In [21]:
```
assert transactions["transaction_id"].equals(
    transactions_raw["transaction_id"]
)

assert sessions["session_id"].equals(
    sessions_raw["session_id"]
)

assert reviews["review_id"].equals(
    reviews_raw["review_id"]
)

print("Record order and IDs match. Safe to restore dates.")
```

```
Record order and IDs match. Safe to restore dates.
```

In [23]:
```
# Converting the original CSV date columns safely

transactions["transaction_date"] = pd.to_datetime(
    transactions_raw["transaction_date"],
    format="mixed",
    dayfirst=True,
    errors="raise"
)

sessions["session_date"] = pd.to_datetime(
    sessions_raw["session_date"],
    format="mixed",
    dayfirst=True,
    errors="raise"
)

reviews["review_date"] = pd.to_datetime(
    reviews_raw["review_date"],
    format="mixed",
    dayfirst=True,
    errors="raise"
)

print("All three date columns converted successfully!")
```

```
All three date columns converted successfully!
```

In [24]:
```
for name, df, col in [
    ("Transactions", transactions, "transaction_date"),
    ("Sessions", sessions, "session_date"),
    ("Reviews", reviews, "review_date")
]:
    print(f"\n{name}")
    print("Data type:", df[col].dtype)
    print("Missing dates:", df[col].isna().sum())
    print("Earliest date:", df[col].min())
    print("Latest date:", df[col].max())
```

```
Transactions
Data type: datetime64[ns]
Missing dates: 0
Earliest date: 2023-01-01 00:23:00
Latest date: 2024-12-30 23:59:00

Sessions
Data type: datetime64[ns]
Missing dates: 0
Earliest date: 2023-01-01 00:24:00
Latest date: 2024-12-30 23:43:00

Reviews
Data type: datetime64[ns]
Missing dates: 0
Earliest date: 2023-01-01 00:00:00
Latest date: 2024-12-30 00:00:00
```

In [25]:
```
# Checking duplicate records across all five datasets

datasets = {
    "Customers": customers,
    "Products": products,
    "Transactions": transactions,
    "Sessions": sessions,
    "Reviews": reviews
```

```
}

for name, df in datasets.items():
    print(f"\n{'=' * 50}")
    print(f"Dataset: {name}")
    print("=" * 50)

    print(f"Total rows: {len(df):,}")
    print(f"Duplicate rows: {df.duplicated().sum():,}")
```

```
==================================================
Dataset: Customers
==================================================
Total rows: 10,000
Duplicate rows: 0

==================================================
Dataset: Products
==================================================
Total rows: 1,000
Duplicate rows: 0

==================================================
Dataset: Transactions
==================================================
Total rows: 120,000
Duplicate rows: 0

==================================================
Dataset: Sessions
==================================================
Total rows: 80,000
Duplicate rows: 0

==================================================
Dataset: Reviews
==================================================
Total rows: 25,000
Duplicate rows: 0
```

In [26]:
```
# Checking primary key uniqueness and missing IDs

primary_keys = {
    "Customers": (customers, "customer_id"),
    "Products": (products, "product_id"),
    "Transactions": (transactions, "transaction_id"),
    "Sessions": (sessions, "session_id"),
    "Reviews": (reviews, "review_id")
}

for name, (df, key) in primary_keys.items():

    print(f"\n{'=' * 50}")
    print(f"Dataset: {name}")
    print("=" * 50)

    if key not in df.columns:
        print(f"Column '{key}' not found. Check the actual column names.")
        continue

    print(f"Missing IDs: {df[key].isna().sum():,}")
    print(f"Duplicate IDs: {df[key].duplicated().sum():,}")
    print(f"Unique IDs: {df[key].nunique():,}")
```

```
==================================================
Dataset: Customers
==================================================
Missing IDs: 0
Duplicate IDs: 0
Unique IDs: 10,000

==================================================
Dataset: Products
==================================================
Missing IDs: 0
Duplicate IDs: 0
Unique IDs: 1,000

==================================================
Dataset: Transactions
==================================================
Missing IDs: 0
Duplicate IDs: 0
Unique IDs: 120,000

==================================================
Dataset: Sessions
==================================================
Missing IDs: 0
Duplicate IDs: 0
Unique IDs: 80,000

==================================================
Dataset: Reviews
==================================================
Missing IDs: 0
Duplicate IDs: 0
Unique IDs: 25,000
```

In [27]:

```python
# Foreign Key Validation

foreign_key_checks = [
    ("Transactions -> Customers",
     transactions, "customer_id",
     customers, "customer_id"),

    ("Sessions -> Customers",
     sessions, "customer_id",
     customers, "customer_id"),

    ("Reviews -> Customers",
     reviews, "customer_id",
     customers, "customer_id"),

    ("Transactions -> Products",
     transactions, "product_id",
     products, "product_id"),

    ("Reviews -> Products",
     reviews, "product_id",
     products, "product_id")
]

for relationship, child_df, child_key, parent_df, parent_key in foreign_key_checks:

    valid_parent_ids = set(parent_df[parent_key].dropna())
```

```
    missing_fk = child_df[child_key].isna()

    invalid_fk = (
        child_df[child_key].notna()
        & ~child_df[child_key].isin(valid_parent_ids)
    )

    print(f"\n{relationship}")
    print(f"  Missing foreign keys: {missing_fk.sum():,}")
    print(f"  Invalid references: {invalid_fk.sum():,}")
    print(
        "  Status:",
        "PASS" if not missing_fk.any() and not invalid_fk.any()
        else "INVESTIGATE"
    )
```

```
Transactions -> Customers
  Missing foreign keys: 0
  Invalid references: 0
  Status: PASS

Sessions -> Customers
  Missing foreign keys: 0
  Invalid references: 0
  Status: PASS

Reviews -> Customers
  Missing foreign keys: 0
  Invalid references: 0
  Status: PASS

Transactions -> Products
  Missing foreign keys: 0
  Invalid references: 0
  Status: PASS

Reviews -> Products
  Missing foreign keys: 0
  Invalid references: 0
  Status: PASS
```

In [28]:

```
# Validating transaction pricing

price_check = transactions.merge(
    products[["product_id", "price"]],
    on="product_id",
    how="left",
    validate="many_to_one"
)

# Calculate expected unit price after discount
price_check["expected_unit_price"] = (
    price_check["price"]
    * (1 - price_check["discount_applied"] / 100)
)

# Calculate difference between recorded and expected prices
price_check["price_difference"] = (
    price_check["unit_price"]
    - price_check["expected_unit_price"]
).abs()

# Allow a small tolerance for floating-point rounding
tolerance = 0.01
```

```
price_mismatches = price_check[
    price_check["price_difference"] > tolerance
]

print("Total transactions:", f"{len(price_check):,}")
print("Price mismatches:", f"{len(price_mismatches):,}")
print(
    "Mismatch percentage:",
    f"{len(price_mismatches) / len(price_check) * 100:.2f}%"
)

if price_mismatches.empty:
    print("Status: PASS — All transaction prices are consistent.")
else:
    print("Status: INVESTIGATE — Some prices do not match.")
    display(price_mismatches.head(10))
```

```
Total transactions: 120,000
Price mismatches: 0
Mismatch percentage: 0.00%
Status: PASS — All transaction prices are consistent.
```

In [29]:
```
# Checking complete duplicate rows in each dataset

for name, df in datasets.items():
    duplicate_count = df.duplicated().sum()
    duplicate_percentage = (duplicate_count / len(df)) * 100

    print(f"\n{name}")
    print("-" * 40)
    print(f"Total rows: {len(df):,}")
    print(f"Duplicate rows: {duplicate_count:,}")
    print(f"Duplicate percentage: {duplicate_percentage:.2f}%")
```

```
Customers
----------------------------------------
Total rows: 10,000
Duplicate rows: 0
Duplicate percentage: 0.00%

Products
----------------------------------------
Total rows: 1,000
Duplicate rows: 0
Duplicate percentage: 0.00%

Transactions
----------------------------------------
Total rows: 120,000
Duplicate rows: 0
Duplicate percentage: 0.00%

Sessions
----------------------------------------
Total rows: 80,000
Duplicate rows: 0
Duplicate percentage: 0.00%

Reviews
----------------------------------------
Total rows: 25,000
Duplicate rows: 0
Duplicate percentage: 0.00%
```

In [30]:
```python
# Validating date ranges

date_columns = {
    "Customers": (customers, "signup_date"),
    "Transactions": (transactions, "transaction_date"),
    "Sessions": (sessions, "session_date"),
    "Reviews": (reviews, "review_date")
}

for name, (df, date_col) in date_columns.items():
    print(f"\n{name}")
    print("-" * 45)
    print(f"Column: {date_col}")
    print(f"Data type: {df[date_col].dtype}")
    print(f"Missing dates: {df[date_col].isna().sum():,}")
    print(f"Earliest date: {df[date_col].min()}")
    print(f"Latest date: {df[date_col].max()}")
```

```
Customers
---------------------------------------------
Column: signup_date
Data type: datetime64[ns]
Missing dates: 0
Earliest date: 2020-01-01 00:00:00
Latest date: 2024-12-30 00:00:00

Transactions
---------------------------------------------
Column: transaction_date
Data type: datetime64[ns]
Missing dates: 0
Earliest date: 2023-01-01 00:23:00
Latest date: 2024-12-30 23:59:00

Sessions
---------------------------------------------
Column: session_date
Data type: datetime64[ns]
Missing dates: 0
Earliest date: 2023-01-01 00:24:00
Latest date: 2024-12-30 23:43:00

Reviews
---------------------------------------------
Column: review_date
Data type: datetime64[ns]
Missing dates: 0
Earliest date: 2023-01-01 00:00:00
Latest date: 2024-12-30 00:00:00
```

In [31]:
```python
# Inspecting original and converted date values

date_checks = [
    (
        "Transactions",
        transactions_raw,
        transactions,
        "transaction_date"
    ),
    (
        "Sessions",
        sessions_raw,
        sessions,
```

```
        "session_date"
    ),
    (
        "Reviews",
        reviews_raw,
        reviews,
        "review_date"
    )
]

for name, raw_df, clean_df, date_col in date_checks:
    print(f"\n{'=' * 60}")
    print(name)
    print("=" * 60)

    comparison = pd.DataFrame({
        "original_date": raw_df[date_col].astype(str),
        "converted_date": clean_df[date_col].astype(str)
    })

    print(comparison.head(10).to_string(index=False))
```

```
============================================================
Transactions
============================================================
   original_date      converted_date
01-01-2023 00:23 2023-01-01 00:23:00
01-01-2023 00:26 2023-01-01 00:26:00
01-01-2023 00:45 2023-01-01 00:45:00
01-01-2023 01:04 2023-01-01 01:04:00
01-01-2023 01:06 2023-01-01 01:06:00
01-01-2023 01:13 2023-01-01 01:13:00
01-01-2023 01:17 2023-01-01 01:17:00
01-01-2023 01:18 2023-01-01 01:18:00
01-01-2023 01:26 2023-01-01 01:26:00
01-01-2023 01:38 2023-01-01 01:38:00

============================================================
Sessions
============================================================
   original_date      converted_date
01-01-2023 00:24 2023-01-01 00:24:00
01-01-2023 00:42 2023-01-01 00:42:00
01-01-2023 00:55 2023-01-01 00:55:00
01-01-2023 01:33 2023-01-01 01:33:00
01-01-2023 01:35 2023-01-01 01:35:00
01-01-2023 01:55 2023-01-01 01:55:00
01-01-2023 02:01 2023-01-01 02:01:00
01-01-2023 02:14 2023-01-01 02:14:00
01-01-2023 02:14 2023-01-01 02:14:00
01-01-2023 02:17 2023-01-01 02:17:00

============================================================
Reviews
============================================================
original_date converted_date
   01-01-2023     2023-01-01
   01-01-2023     2023-01-01
   01-01-2023     2023-01-01
   01-01-2023     2023-01-01
   01-01-2023     2023-01-01
   01-01-2023     2023-01-01
   01-01-2023     2023-01-01
   01-01-2023     2023-01-01
   01-01-2023     2023-01-01
   01-01-2023     2023-01-01
```

In [32]:

```python
# Inspecting dates where day-first and month-first interpretations differ

for name, raw_df, clean_df, date_col in date_checks:
    print(f"\n{'=' * 65}")
    print(name)
    print("=" * 65)

    original = raw_df[date_col].astype(str).str.strip()

    # Look for DD-MM-YYYY or MM-DD-YYYY patterns
    parts = original.str.extract(
        r"^(\d{2})-(\d{2})-(\d{4})"
    )

    first = pd.to_numeric(parts[0], errors="coerce")
    second = pd.to_numeric(parts[1], errors="coerce")

    # Dates with a component > 12 help distinguish the formats
```

```
    mask = (
        ((first > 12) & (second <= 12)) |
        ((second > 12) & (first <= 12))
    )

    sample = pd.DataFrame({
        "original_date": original[mask],
        "converted_date": clean_df.loc[mask, date_col].astype(str)
    })

    print(f"Distinguishing date examples: {len(sample):,}")
    print(sample.head(10).to_string(index=False))
```

```
======================================================================
Transactions
======================================================================
Distinguishing date examples: 72,506
   original_date      converted_date
13-01-2023 00:00 2023-01-13 00:00:00
13-01-2023 00:28 2023-01-13 00:28:00
13-01-2023 00:30 2023-01-13 00:30:00
13-01-2023 00:40 2023-01-13 00:40:00
13-01-2023 01:00 2023-01-13 01:00:00
13-01-2023 01:03 2023-01-13 01:03:00
13-01-2023 01:17 2023-01-13 01:17:00
13-01-2023 01:18 2023-01-13 01:18:00
13-01-2023 01:21 2023-01-13 01:21:00
13-01-2023 01:32 2023-01-13 01:32:00

======================================================================
Sessions
======================================================================
Distinguishing date examples: 48,498
   original_date      converted_date
13-01-2023 00:47 2023-01-13 00:47:00
13-01-2023 00:51 2023-01-13 00:51:00
13-01-2023 01:31 2023-01-13 01:31:00
13-01-2023 01:31 2023-01-13 01:31:00
13-01-2023 01:41 2023-01-13 01:41:00
13-01-2023 01:50 2023-01-13 01:50:00
13-01-2023 01:55 2023-01-13 01:55:00
13-01-2023 02:02 2023-01-13 02:02:00
13-01-2023 02:28 2023-01-13 02:28:00
13-01-2023 03:03 2023-01-13 03:03:00

======================================================================
Reviews
======================================================================
Distinguishing date examples: 15,174
original_date converted_date
   13-01-2023     2023-01-13
   13-01-2023     2023-01-13
   13-01-2023     2023-01-13
   13-01-2023     2023-01-13
   13-01-2023     2023-01-13
   13-01-2023     2023-01-13
   13-01-2023     2023-01-13
   13-01-2023     2023-01-13
   13-01-2023     2023-01-13
   13-01-2023     2023-01-13
```

In [33]:
```
# Counting date-format patterns in the original CSVs

for name, raw_df, clean_df, date_col in date_checks:
```

```
    original = raw_df[date_col].astype(str).str.strip()

    parts = original.str.extract(r"^(\d{2})-(\d{2})-(\d{4})")

    first = pd.to_numeric(parts[0], errors="coerce")
    second = pd.to_numeric(parts[1], errors="coerce")

    day_first_evidence = (first > 12).sum()
    month_first_evidence = (second > 12).sum()
    ambiguous = (
        (first >= 1) & (first <= 12) &
        (second >= 1) & (second <= 12)
    ).sum()

    print(f"\n{name}")
    print("-" * 45)
    print(f"DD-MM evidence: {day_first_evidence:,}")
    print(f"MM-DD evidence: {month_first_evidence:,}")
    print(f"Ambiguous dates: {ambiguous:,}")
    print(f"Other/unmatched formats: {original.str.extract(r'^(.*)$')[0].isna().sum():,}")
```

```
Transactions
---------------------------------------------
DD-MM evidence: 72,506
MM-DD evidence: 0
Ambiguous dates: 47,494
Other/unmatched formats: 0

Sessions
---------------------------------------------
DD-MM evidence: 48,498
MM-DD evidence: 0
Ambiguous dates: 31,502
Other/unmatched formats: 0

Reviews
---------------------------------------------
DD-MM evidence: 15,174
MM-DD evidence: 0
Ambiguous dates: 9,826
Other/unmatched formats: 0
```

In [34]:
```
# Validating activity dates against expected boundaries

start_date = pd.Timestamp("2023-01-01")
end_date = pd.Timestamp("2024-12-30 23:59:59")

activity_dates = {
    "Transactions": transactions["transaction_date"],
    "Sessions": sessions["session_date"],
    "Reviews": reviews["review_date"]
}

for name, dates in activity_dates.items():
    before_start = (dates < start_date).sum()
    after_end = (dates > end_date).sum()
    missing = dates.isna().sum()

    print(f"\n{name}")
    print("-" * 45)
    print(f"Dates before expected range: {before_start:,}")
    print(f"Dates after expected range: {after_end:,}")
    print(f"Missing dates: {missing:,}")
```

```
    if before_start == 0 and after_end == 0 and missing == 0:
        print("Status: PASS — All dates fall within the expected range.")
    else:
        print("Status: REVIEW — Investigate date values.")
```

```
Transactions
--------------------------------------------
Dates before expected range: 0
Dates after expected range: 0
Missing dates: 0
Status: PASS — All dates fall within the expected range.

Sessions
--------------------------------------------
Dates before expected range: 0
Dates after expected range: 0
Missing dates: 0
Status: PASS — All dates fall within the expected range.

Reviews
--------------------------------------------
Dates before expected range: 0
Dates after expected range: 0
Missing dates: 0
Status: PASS — All dates fall within the expected range.
```

In [35]:
```
# Checking whether activity occurred before customer signup

customer_signup = customers[["customer_id", "signup_date"]]

# Check transactions
transaction_check = transactions.merge(
    customer_signup,
    on="customer_id",
    how="left",
    validate="many_to_one"
)

transactions_before_signup = transaction_check[
    transaction_check["transaction_date"]
    < transaction_check["signup_date"]
]

print("TRANSACTIONS")
print("-" * 45)
print(f"Total transactions: {len(transaction_check):,}")
print(f"Transactions before signup: {len(transactions_before_signup):,}")

# Check sessions
session_check = sessions.merge(
    customer_signup,
    on="customer_id",
    how="left",
    validate="many_to_one"
)

sessions_before_signup = session_check[
    session_check["session_date"]
    < session_check["signup_date"]
]

print("\nSESSIONS")
print("-" * 45)
```

```
print(f"Total sessions: {len(session_check):,}")
print(f"Sessions before signup: {len(sessions_before_signup):,}")
```

```
TRANSACTIONS
--------------------------------------------
Total transactions: 120,000
Transactions before signup: 11,027

SESSIONS
--------------------------------------------
Total sessions: 80,000
Sessions before signup: 7,294
```

In [36]:

```
#  Measuring how far activities occur before signup

# Transactions
transaction_check["days_before_signup"] = (
    transaction_check["signup_date"]
    - transaction_check["transaction_date"]
).dt.total_seconds() / (24 * 60 * 60)

transactions_before_signup = transaction_check[
    transaction_check["days_before_signup"] > 0
]

print("TRANSACTIONS BEFORE SIGNUP")
print("-" * 50)

if len(transactions_before_signup) > 0:
    print(f"Count: {len(transactions_before_signup):,}")
    print(
        "Minimum days before signup:",
        round(transactions_before_signup["days_before_signup"].min(), 2)
    )
    print(
        "Median days before signup:",
        round(transactions_before_signup["days_before_signup"].median(), 2)
    )
    print(
        "Maximum days before signup:",
        round(transactions_before_signup["days_before_signup"].max(), 2)
    )

# Sessions
session_check["days_before_signup"] = (
    session_check["signup_date"]
    - session_check["session_date"]
).dt.total_seconds() / (24 * 60 * 60)

sessions_before_signup = session_check[
    session_check["days_before_signup"] > 0
]

print("\nSESSIONS BEFORE SIGNUP")
print("-" * 50)

if len(sessions_before_signup) > 0:
    print(f"Count: {len(sessions_before_signup):,}")
    print(
        "Minimum days before signup:",
        round(sessions_before_signup["days_before_signup"].min(), 2)
    )
    print(
        "Median days before signup:",
```

```
        round(sessions_before_signup["days_before_signup"].median(), 2)
    )
    print(
        "Maximum days before signup:",
        round(sessions_before_signup["days_before_signup"].max(), 2)
    )
```

```
TRANSACTIONS BEFORE SIGNUP
--------------------------------------------------
Count: 11,027
Minimum days before signup: 0.02
Median days before signup: 145.01
Maximum days before signup: 695.36

SESSIONS BEFORE SIGNUP
--------------------------------------------------
Count: 7,294
Minimum days before signup: 0.01
Median days before signup: 145.21
Maximum days before signup: 698.03
```

In [37]:

```
# Identifying customers with pre-signup activity

transaction_customer_ids = set(
    transactions_before_signup["customer_id"]
)

session_customer_ids = set(
    sessions_before_signup["customer_id"]
)

print("CUSTOMER-LEVEL INVESTIGATION")
print("-" * 50)

print(
    "Customers with transactions before signup:",
    f"{len(transaction_customer_ids):,}"
)

print(
    "Customers with sessions before signup:",
    f"{len(session_customer_ids):,}"
)

print(
    "Customers with both issues:",
    f"{len(transaction_customer_ids & session_customer_ids):,}"
)

print(
    "Customers with only transaction issue:",
    f"{len(transaction_customer_ids - session_customer_ids):,}"
)

print(
    "Customers with only session issue:",
    f"{len(session_customer_ids - transaction_customer_ids):,}"
)
```

```
CUSTOMER-LEVEL INVESTIGATION
--------------------------------------------------
Customers with transactions before signup: 3,035
Customers with sessions before signup: 2,707
Customers with both issues: 2,421
Customers with only transaction issue: 614
Customers with only session issue: 286
```

In [38]:

```python
# Inspecting inconsistent customer records

# Customers with both transaction and session issues
customers_with_both = (
    transaction_customer_ids & session_customer_ids
)

# Select a small sample of 10 customers
sample_customer_ids = list(sorted(customers_with_both))[:10]

# Customer signup dates
sample_customers = customers[
    customers["customer_id"].isin(sample_customer_ids)
][["customer_id", "signup_date"]]

# Their earliest transactions
sample_transactions = (
    transactions[
        transactions["customer_id"].isin(sample_customer_ids)
    ]
    .groupby("customer_id")["transaction_date"]
    .min()
    .reset_index(name="earliest_transaction")
)

# Their earliest sessions
sample_sessions = (
    sessions[
        sessions["customer_id"].isin(sample_customer_ids)
    ]
    .groupby("customer_id")["session_date"]
    .min()
    .reset_index(name="earliest_session")
)

# Combine the information
sample_check = (
    sample_customers
    .merge(sample_transactions, on="customer_id", how="left")
    .merge(sample_sessions, on="customer_id", how="left")
)

print(sample_check.to_string(index=False))
```

```
customer_id signup_date earliest_transaction    earliest_session
     C00014  2024-06-03  2023-07-30 23:30:00 2023-01-22 06:03:00
     C00019  2023-07-15  2023-01-11 11:23:00 2023-02-16 06:38:00
     C00027  2024-04-08  2023-10-24 06:39:00 2023-07-30 11:46:00
     C00028  2024-03-02  2023-09-02 19:08:00 2023-10-18 09:11:00
     C00030  2023-06-18  2023-02-15 08:56:00 2023-03-12 12:39:00
     C00033  2023-10-09  2023-01-11 15:07:00 2023-01-13 12:06:00
     C00037  2023-07-15  2023-01-27 18:15:00 2023-01-05 14:45:00
     C00038  2023-05-22  2023-02-01 08:48:00 2023-01-22 08:54:00
     C00040  2024-04-12  2023-01-19 05:32:00 2023-03-01 00:22:00
     C00045  2023-06-25  2023-04-11 06:35:00 2023-02-25 13:24:00
```

In [39]:

```
#  Comparing customer signup dates with earliest activity

# Earliest transaction per customer
earliest_transactions = (
    transactions.groupby("customer_id")["transaction_date"]
    .min()
    .reset_index(name="earliest_transaction")
)

# Earliest session per customer
earliest_sessions = (
    sessions.groupby("customer_id")["session_date"]
    .min()
    .reset_index(name="earliest_session")
)

# Combine with customer signup dates
customer_timeline = (
    customers[["customer_id", "signup_date"]]
    .merge(earliest_transactions, on="customer_id", how="left")
    .merge(earliest_sessions, on="customer_id", how="left")
)

# Calculate time differences
customer_timeline["transaction_days_before_signup"] = (
    customer_timeline["signup_date"]
    - customer_timeline["earliest_transaction"]
).dt.total_seconds() / 86400

customer_timeline["session_days_before_signup"] = (
    customer_timeline["signup_date"]
    - customer_timeline["earliest_session"]
).dt.total_seconds() / 86400

print("CUSTOMER TIMELINE SUMMARY")
print("-" * 55)

print(
    "Customers with transactions before signup:",
    (customer_timeline["transaction_days_before_signup"] > 0).sum()
)

print(
    "Customers with sessions before signup:",
    (customer_timeline["session_days_before_signup"] > 0).sum()
)

print(
    "Customers with both activities before signup:",
    (
        (customer_timeline["transaction_days_before_signup"] > 0)
        & (customer_timeline["session_days_before_signup"] > 0)
    ).sum()
)

print("\nMedian days between earliest transaction and signup:")
print(
    round(
        customer_timeline.loc[
            customer_timeline["transaction_days_before_signup"] > 0,
            "transaction_days_before_signup"
        ].median(),
        2
```

```
    )
)

print("\nMedian days between earliest session and signup:")
print(
    round(
        customer_timeline.loc[
            customer_timeline["session_days_before_signup"] > 0,
            "session_days_before_signup"
        ].median(),
        2
    )
)
```

```
CUSTOMER TIMELINE SUMMARY
----------------------------------------------------------
Customers with transactions before signup: 3035
Customers with sessions before signup: 2707
Customers with both activities before signup: 2421

Median days between earliest transaction and signup:
204.66

Median days between earliest session and signup:
195.15
```

In [41]:
```
print("TRANSACTION PRICE CHECK")
print("Negative prices:", (transactions["unit_price"] < 0).sum())
print("Invalid discounts:", (
    (transactions["discount_applied"] < 0) |
    (transactions["discount_applied"] > 100)
).sum())

print("\nPRODUCT PRICE CHECK")
print("Non-positive prices:", (products["price"] <= 0).sum())

print("\nREVIEW RATING CHECK")
print("Ratings below 1:", (reviews["rating"] < 1).sum())
print("Ratings above 5:", (reviews["rating"] > 5).sum())
```

```
TRANSACTION PRICE CHECK
Negative prices: 0
Invalid discounts: 0

PRODUCT PRICE CHECK
Non-positive prices: 0

REVIEW RATING CHECK
Ratings below 1: 0
Ratings above 5: 0
```

In [42]:
```
print(transactions["status"].value_counts(dropna=False))

print("\nStatus percentages:")
print(
    (transactions["status"].value_counts(normalize=True, dropna=False) * 100)
    .round(2)
)
```

```
status
completed    68700
refunded     17139
cancelled    17092
pending      17069
Name: count, dtype: int64

Status percentages:
status
completed   57.25
refunded    14.28
cancelled   14.24
pending     14.22
Name: proportion, dtype: float64
```

In [43]:
```python
# Transaction value by status

transactions["transaction_value"] = (
    transactions["unit_price"] * transactions["quantity"]
)

status_revenue = (
    transactions.groupby("status")
    .agg(
        transaction_count=("transaction_id", "count"),
        total_value=("transaction_value", "sum"),
        average_value=("transaction_value", "mean")
    )
    .round(2)
    .sort_values("total_value", ascending=False)
)

print(status_revenue)
```

```
           transaction_count  total_value  average_value
status
completed              68700 5,443,245.13          79.23
cancelled              17092 1,339,251.68          78.36
pending                17069 1,316,712.31          77.14
refunded               17139 1,314,563.25          76.70
```

In [44]:
```python
# Validating transaction quantities

print("TRANSACTION QUANTITY CHECK")
print("-" * 40)

print("Zero quantity:", (transactions["quantity"] == 0).sum())
print("Negative quantity:", (transactions["quantity"] < 0).sum())
print("Missing quantity:", transactions["quantity"].isna().sum())

print("\nQuantity summary:")
print(transactions["quantity"].describe())
```

```
TRANSACTION QUANTITY CHECK
----------------------------------------
Zero quantity: 0
Negative quantity: 0
Missing quantity: 0

Quantity summary:
count   120,000.00
mean          1.47
std           1.14
min           1.00
25%           1.00
50%           1.00
75%           1.00
max          19.00
Name: quantity, dtype: float64
```

In [48]:
```python
# Inspecting categorical values

categorical_columns = {
    "Customers": (customers, ["gender", "segment"]),
    "Products": (products, ["category", "brand"]),
    "Transactions": (transactions, ["status"]),
    "Sessions": (sessions, ["device", "channel"]),
}

for name, (df, columns) in categorical_columns.items():
    print(f"\n{'=' * 45}\n{name}\n{'=' * 45}")

    for col in columns:
        print(f"\n{col}:")
        print(df[col].value_counts(dropna=False).to_string())
```

```
==========================================
Customers
==========================================

gender:
gender
F                     4563
M                     4466
Prefer not to say      495
Non-binary             476

segment:
segment
Regular               3500
Budget Shopper        3043
Premium               1982
VIP                    974
Occasional Visitor     501

==========================================
Products
==========================================

category:
category
Health              91
Sports              75
Food & Grocery      75
Electronics         73
Office Supplies     73
Jewelry             72
Clothing            71
Automotive          69
Pet Supplies        68
Home & Garden       67
Toys                60
Software            54
Music               52
Beauty              50
Books               50

brand:
brand
PureBrand          64
EcoLiving          59
PetLife            58
GlowUp             57
StyleMax           56
FitGear            55
NovaTech           54
ShineOn            53
HomeEssentials     52
PlayTime           52
SoundWave          50
DigiTools          49
UrbanStyle         49
ClassicCo          48
FreshMarket        44
TechPro            42
WellBeing          42
WorkSmart          40
ReadMore           39
AutoParts          37
```

```
==========================================
Transactions
==========================================

status:
status
completed    68700
refunded     17139
cancelled    17092
pending      17069

==========================================
Sessions
==========================================

device:
device
mobile     40010
desktop    28053
tablet     11937

channel:
channel
organic        19974
paid_search    15915
social         12071
direct         12067
email          11854
referral        8119
```

In [49]:

```python
# Inspecting Boolean-like columns

boolean_columns = {
    "Customers": (customers, ["is_churned", "email_opt_in", "has_app"]),
    "Products": (products, ["is_featured"]),
    "Sessions": (sessions, ["converted", "bounced"])
}

for name, (df, columns) in boolean_columns.items():
    print(f"\n{name}")
    print("-" * 40)

    for col in columns:
        print(f"\n{col}:")
        print(df[col].value_counts(dropna=False).to_string())
```

```
Customers
----------------------------------------

is_churned:
is_churned
0    8306
1    1694

email_opt_in:
email_opt_in
1    7005
0    2995

has_app:
has_app
1    5572
0    4428

Products
----------------------------------------

is_featured:
is_featured
0    897
1    103

Sessions
----------------------------------------

converted:
converted
0    71352
1     8648

bounced:
bounced
0    72746
1     7254
```

Table Joins

In [50]:
```python
# Checking the available columns before joining
for name, df in datasets.items():
    print(f"{name}: {df.shape}")
    print(list(df.columns))
    print()
```

```
Customers: (10000, 10)
['customer_id', 'signup_date', 'age', 'gender', 'country', 'segment', 'is_churned', 'lifetime_
value', 'email_opt_in', 'has_app']

Products: (1000, 11)
['product_id', 'product_name', 'category', 'brand', 'price', 'avg_rating', 'num_ratings', 'sto
ck_quantity', 'discount_pct', 'is_featured', 'weight_kg']

Transactions: (120000, 12)
['transaction_id', 'customer_id', 'product_id', 'transaction_date', 'quantity', 'unit_price',
'total_amount', 'discount_applied', 'status', 'payment_method', 'shipping_cost', 'transaction_
value']

Sessions: (80000, 10)
['session_id', 'customer_id', 'session_date', 'device', 'channel', 'duration_seconds', 'pages_
viewed', 'converted', 'bounced', 'cart_additions']

Reviews: (25000, 8)
['review_id', 'customer_id', 'product_id', 'review_date', 'rating', 'review_text', 'helpful_vo
tes', 'verified_purchase']
```

In [51]:
```
# Aggregate transaction metrics at customer level
customer_transactions = transactions.groupby("customer_id").agg(
    total_transactions=("transaction_id", "nunique"),
    total_quantity=("quantity", "sum"),
    total_transaction_value=("transaction_value", "sum"),
    avg_order_value=("transaction_value", "mean"),
    first_transaction_date=("transaction_date", "min"),
    last_transaction_date=("transaction_date", "max")
).reset_index()

print("Customer transaction metrics:", customer_transactions.shape)
customer_transactions.head()
```

```
Customer transaction metrics: (9941, 7)
```

Out[51]:

	customer_id	total_transactions	total_quantity	total_transaction_value	avg_order_value	first_transac
0	C00000	25	32	970.60	38.82	2023-05-1
1	C00001	12	22	786.15	65.51	2023-03-3
2	C00002	29	39	2,112.48	72.84	2023-01-2
3	C00003	26	40	1,341.04	51.58	2023-01-0
4	C00004	4	4	184.54	46.14	2023-03-2

In [52]:
```
customer_sessions = sessions.groupby("customer_id").agg(
    total_sessions=("session_id", "nunique"),
    avg_session_duration=("duration_seconds", "mean"),
    avg_pages_viewed=("pages_viewed", "mean"),
    total_cart_additions=("cart_additions", "sum"),
    converted_sessions=("converted", "sum"),
    bounced_sessions=("bounced", "sum")
).reset_index()

print("Customer session metrics:", customer_sessions.shape)
customer_sessions.head()
```

```
Customer session metrics: (9824, 7)
```

Out[52]:

	customer_id	total_sessions	avg_session_duration	avg_pages_viewed	total_cart_additions	converted
0	C00000	8	696.88	10.75	5	
1	C00001	12	325.50	6.83	4	
2	C00002	29	439.90	7.48	31	
3	C00003	17	284.59	5.18	20	
4	C00004	1	216.00	5.00	0	

In [53]:
```
customer_reviews = reviews.groupby("customer_id").agg(
    total_reviews=("review_id", "nunique"),
    avg_review_rating=("rating", "mean"),
    total_helpful_votes=("helpful_votes", "sum"),
    verified_reviews=("verified_purchase", "sum")
).reset_index()

print("Customer review metrics:", customer_reviews.shape)
customer_reviews.head()
```

```
Customer review metrics: (9174, 5)
```

Out[53]:

	customer_id	total_reviews	avg_review_rating	total_helpful_votes	verified_reviews
0	C00001	1	2.00	41	1
1	C00002	2	2.50	42	1
2	C00003	1	4.00	25	1
3	C00004	1	2.00	15	1
4	C00005	3	3.33	53	3

In [54]:
```
## Creating Customer Master Dataset
# Starting with all customers, then add aggregated metrics
customer_master = (
    customers
    .merge(customer_transactions, on="customer_id", how="left")
    .merge(customer_sessions, on="customer_id", how="left")
    .merge(customer_reviews, on="customer_id", how="left")
)

# Fill zero only for count metrics where no activity record exists
count_columns = [
    "total_transactions", "total_quantity",
    "total_sessions", "total_cart_additions",
    "converted_sessions", "bounced_sessions",
    "total_reviews", "total_helpful_votes", "verified_reviews"
]

customer_master[count_columns] = (
    customer_master[count_columns].fillna(0)
)

print("Customer master shape:", customer_master.shape)
print("Unique customers:", customer_master["customer_id"].nunique())
print("Duplicate customer IDs:", customer_master["customer_id"].duplicated().sum())

customer_master.head()
```

```
Customer master shape: (10000, 26)
Unique customers: 10000
Duplicate customer IDs: 0
```

Out[54]:

	customer_id	signup_date	age	gender	country	segment	is_churned	lifetime_value	email_opt_in
0	C00000	2020-02-13	28	M	BR	Premium	0	1,595.27	0
1	C00001	2021-10-01	22	Prefer not to say	FR	Regular	0	1,160.61	1
2	C00002	2022-06-26	30	F	US	VIP	0	3,093.32	1
3	C00003	2021-12-21	48	F	US	Premium	1	2,131.08	1
4	C00004	2021-09-16	37	M	CA	Budget Shopper	0	583.39	0

In [55]:

```
# Checking missing values in customer master
missing_summary = customer_master.isna().sum()
print("Columns with missing values:")
print(missing_summary[missing_summary > 0].to_string())

# Confirming activity counts are not missing
count_columns = [
    "total_transactions", "total_quantity",
    "total_sessions", "total_cart_additions",
    "converted_sessions", "bounced_sessions",
    "total_reviews", "total_helpful_votes", "verified_reviews"
]

print("\nMissing values in count columns:",
      customer_master[count_columns].isna().sum().sum())
```

```
Columns with missing values:
total_transaction_value     59
avg_order_value             59
first_transaction_date      59
last_transaction_date       59
avg_session_duration       176
avg_pages_viewed           176
avg_review_rating          826

Missing values in count columns: 0
```

RFM Customer Segmentation

In [56]:

```
# Checking transaction status distribution
print(transactions["status"].value_counts(dropna=False))

# Summarize transaction value by status
print(
    transactions.groupby("status")["transaction_value"]
    .agg(["count", "sum", "mean"])
    .round(2)
)
```

```
status
completed    68700
refunded     17139
cancelled    17092
pending      17069
Name: count, dtype: int64
           count          sum  mean
status
cancelled  17092 1,339,251.68 78.36
completed  68700 5,443,245.13 79.23
pending    17069 1,316,712.31 77.14
refunded   17139 1,314,563.25 76.70
```

In [57]:
```python
# Using completed transactions only for initial RFM
completed = transactions.loc[
    transactions["status"].eq("completed")
].copy()

reference_date = transactions["transaction_date"].max().normalize() + pd.Timedelta(days=1)

rfm = completed.groupby("customer_id").agg(
    last_purchase=("transaction_date", "max"),
    frequency=("transaction_id", "nunique"),
    monetary=("transaction_value", "sum")
).reset_index()

rfm["recency"] = (
    reference_date - rfm["last_purchase"].dt.normalize()
).dt.days

rfm = rfm[["customer_id", "recency", "frequency", "monetary"]]

print("Reference date:", reference_date.date())
print("RFM customers:", rfm.shape[0])
print(rfm[["recency", "frequency", "monetary"]].describe().round(2))
```

```
Reference date: 2024-12-31
RFM customers: 9730
       recency  frequency  monetary
count 9,730.00   9,730.00  9,730.00
mean    131.01       7.06    559.43
std     139.85       4.68    511.40
min       1.00       1.00      2.68
25%      33.00       4.00    200.00
50%      82.00       6.00    421.85
75%     177.00      10.00    761.21
max     730.00      30.00  7,346.48
```

In [59]:
```python
## Calculating RFM scores
# Recency: lower days = higher score

# Recency: lower days = higher score
rfm["R_score"] = pd.qcut(
    rfm["recency"].rank(method="first"),
    5, labels=[5, 4, 3, 2, 1]
).astype(int)

# Frequency and Monetary: higher values = higher scores
rfm["F_score"] = pd.qcut(
    rfm["frequency"].rank(method="first"),
    5, labels=[1, 2, 3, 4, 5]
).astype(int)

rfm["M_score"] = pd.qcut(
```

```
    rfm["monetary"].rank(method="first"),
    5, labels=[1, 2, 3, 4, 5]
).astype(int)

rfm["RFM_score"] = (
    rfm["R_score"].astype(str)
    + rfm["F_score"].astype(str)
    + rfm["M_score"].astype(str)
)

print(rfm[["R_score", "F_score", "M_score"]].describe().round(2))
print("\nSample:")
print(rfm.head())
```

```
       R_score  F_score  M_score
count 9,730.00 9,730.00 9,730.00
mean       3.00     3.00     3.00
std        1.41     1.41     1.41
min        1.00     1.00     1.00
25%        2.00     2.00     2.00
50%        3.00     3.00     3.00
75%        4.00     4.00     4.00
max        5.00     5.00     5.00

Sample:
  customer_id  recency  frequency  monetary  R_score  F_score  M_score  \
0      C00000        5         18    706.65        5        5        4
1      C00001        8          5    244.63        5        2        2
2      C00002      171         16  1,104.66        2        5        5
3      C00003        8         15    695.06        5        5        4
4      C00004      258          2    148.32        1        1        1

  RFM_score
0       554
1       522
2       255
3       554
4       111
```

Assign Customer Segments

In [60]:

```
def assign_segment(row):
    r, f, m = row["R_score"], row["F_score"], row["M_score"]

    if r >= 4 and f >= 4 and m >= 4:
        return "Best Customers"
    elif f >= 4 and r >= 3:
        return "Loyal Customers"
    elif r >= 4 and f in [2, 3]:
        return "Potential Loyalists"
    elif r <= 2 and f >= 3:
        return "At Risk"
    elif r <= 2 and f <= 2:
        return "Need Attention"
    else:
        return "Other Customers"

rfm["customer_segment"] = rfm.apply(assign_segment, axis=1)

print(rfm["customer_segment"].value_counts())
print("\nSegment percentages:")
print((rfm["customer_segment"].value_counts(normalize=True) * 100).round(2))
```

```
customer_segment
Need Attention          2447
Best Customers          1874
At Risk                 1445
Other Customers         1337
Loyal Customers         1326
Potential Loyalists     1301
Name: count, dtype: int64

Segment percentages:
customer_segment
Need Attention         25.15
Best Customers         19.26
At Risk                14.85
Other Customers        13.74
Loyal Customers        13.63
Potential Loyalists    13.37
Name: proportion, dtype: float64
```

In [62]:
```python
# Selecting the RFM columns to merge
rfm_columns = [
    "customer_id", "recency", "frequency", "monetary",
    "R_score", "F_score", "M_score", "RFM_score",
    "customer_segment"
]

customer_master = customer_master.merge(
    rfm[rfm_columns],
    on="customer_id",
    how="left",
    validate="one_to_one"
)

# Identify customers without completed transactions
customer_master["purchase_group"] = np.where(
    customer_master["RFM_score"].isna(),
    "No Completed Purchase",
    "Has Completed Purchase"
)

print("Master shape:", customer_master.shape)
print("Unique customers:", customer_master["customer_id"].nunique())
print("\nPurchase groups:")
print(customer_master["purchase_group"].value_counts())
```

```
Master shape: (10000, 35)
Unique customers: 10000

Purchase groups:
purchase_group
Has Completed Purchase    9730
No Completed Purchase      270
Name: count, dtype: int64
```

Cohort Analysis

Prepare the Cohort Data

In [63]:
```python
# Creating signup cohort month
customers["cohort_month"] = (
    customers["signup_date"]
```

```
    .dt.to_period("M")
    .dt.to_timestamp()
)

# Create session activity month
sessions_cohort = sessions[
    ["customer_id", "session_date"]
].copy()

sessions_cohort["activity_month"] = (
    sessions_cohort["session_date"]
    .dt.to_period("M")
    .dt.to_timestamp()
)

# Attach signup cohort to session activity
sessions_cohort = sessions_cohort.merge(
    customers[["customer_id", "cohort_month"]],
    on="customer_id",
    how="left",
    validate="many_to_one"
)

print("Session records:", len(sessions_cohort))
print("Missing cohort months:", sessions_cohort["cohort_month"].isna().sum())
print(sessions_cohort.head())
```

```
Session records: 80000
Missing cohort months: 0
  customer_id        session_date activity_month cohort_month
0      C03202 2023-01-01 00:24:00     2023-01-01   2022-01-01
1      C09526 2023-01-01 00:42:00     2023-01-01   2022-08-01
2      C07034 2023-01-01 00:55:00     2023-01-01   2022-07-01
3      C05327 2023-01-01 01:33:00     2023-01-01   2022-09-01
4      C05338 2023-01-01 01:35:00     2023-01-01   2022-10-01
```

Cleaning Cohort Activity

In [64]:

```
# Attaching actual signup dates to session records
sessions_cohort = sessions_cohort.merge(
    customers[["customer_id", "signup_date"]],
    on="customer_id",
    how="left",
    validate="many_to_one"
)

# Identify sessions before signup
pre_signup = sessions_cohort["session_date"] < sessions_cohort["signup_date"]

print("Sessions before signup:", pre_signup.sum())
print("Valid session records:", (~pre_signup).sum())

# Keep sessions on or after signup
sessions_cohort_clean = sessions_cohort.loc[~pre_signup].copy()

# Calculate cohort age in months
sessions_cohort_clean["cohort_index"] = (
    (sessions_cohort_clean["activity_month"].dt.year -
     sessions_cohort_clean["cohort_month"].dt.year) * 12
    + sessions_cohort_clean["activity_month"].dt.month
    - sessions_cohort_clean["cohort_month"].dt.month
)
```

```
print("Negative cohort indexes:",
      (sessions_cohort_clean["cohort_index"] < 0).sum())
```

```
Sessions before signup: 7294
Valid session records: 72706
Negative cohort indexes: 0
```

Building the Cohort Retention Matrix

In [65]:
```
# Unique active customers per cohort and activity month
cohort_activity = sessions_cohort_clean.groupby(
    ["cohort_month", "cohort_index"]
)["customer_id"].nunique().reset_index(name="active_customers")

# Cohort size: unique customers with valid observed activity
cohort_sizes = (
    sessions_cohort_clean.groupby("cohort_month")["customer_id"]
    .nunique()
    .rename("cohort_size")
)

# Pivot into a retention table
cohort_counts = cohort_activity.pivot(
    index="cohort_month",
    columns="cohort_index",
    values="active_customers"
)

# Divide by cohort size
cohort_retention = cohort_counts.div(cohort_sizes, axis=0) * 100

print("Cohort retention matrix (%):")
print(cohort_retention.round(1).iloc[:12, :12])
```

```
Cohort retention matrix (%):
cohort_index  0   1   2   3   4   5   6   7   8   9   10  11
cohort_month
2020-01-01   NaN NaN NaN NaN NaN NaN NaN NaN NaN NaN NaN NaN
2020-02-01   NaN NaN NaN NaN NaN NaN NaN NaN NaN NaN NaN NaN
2020-03-01   NaN NaN NaN NaN NaN NaN NaN NaN NaN NaN NaN NaN
2020-04-01   NaN NaN NaN NaN NaN NaN NaN NaN NaN NaN NaN NaN
2020-05-01   NaN NaN NaN NaN NaN NaN NaN NaN NaN NaN NaN NaN
2020-06-01   NaN NaN NaN NaN NaN NaN NaN NaN NaN NaN NaN NaN
2020-07-01   NaN NaN NaN NaN NaN NaN NaN NaN NaN NaN NaN NaN
2020-08-01   NaN NaN NaN NaN NaN NaN NaN NaN NaN NaN NaN NaN
2020-09-01   NaN NaN NaN NaN NaN NaN NaN NaN NaN NaN NaN NaN
2020-10-01   NaN NaN NaN NaN NaN NaN NaN NaN NaN NaN NaN NaN
2020-11-01   NaN NaN NaN NaN NaN NaN NaN NaN NaN NaN NaN NaN
2020-12-01   NaN NaN NaN NaN NaN NaN NaN NaN NaN NaN NaN NaN
```

Correct Cohort Retention Matrix

In [66]:
```
# Cohort size = all customers who signed up in each month
cohort_sizes = (
    customers.groupby("cohort_month")["customer_id"]
    .nunique()
)

# Active customers per signup cohort and activity month
cohort_activity = sessions_cohort_clean.groupby(
```

```
    ["cohort_month", "cohort_index"]
)["customer_id"].nunique()

# Pivot into a matrix
cohort_counts = cohort_activity.unstack()

# Divide by all customers in each signup cohort
cohort_retention = (
    cohort_counts.div(cohort_sizes, axis=0) * 100
)

# Show cohorts with signup dates from Jan 2023 onward
cohort_retention_recent = cohort_retention.loc[
    cohort_retention.index >= "2023-01-01"
]

print(cohort_retention_recent.iloc[:, :12].round(1))
```

```
cohort_index    0     1     2     3     4     5     6     7     8     9   \
cohort_month
2023-01-01   17.40 26.10 26.10 31.30 30.00 28.70 29.60 27.00 23.90 27.00
2023-02-01   15.40 26.30 26.30 30.70 32.50 25.00 29.80 33.30 31.10 32.50
2023-03-01   14.60 29.20 30.80 26.20 28.70 23.30 30.80 25.40 28.30 30.80
2023-04-01   14.20 28.00 21.10 27.10 27.50 26.60 24.30 29.80 31.70 29.40
2023-05-01   16.50 26.60 28.90 25.70 24.80 29.80 32.60 26.60 29.40 33.90
2023-06-01   12.50 27.30 27.00 22.70 27.00 26.60 27.30 22.70 23.00 31.60
2023-07-01    9.00 17.40 15.20 19.70 13.50 20.20 16.30 16.90 13.50 20.80
2023-08-01   10.30 20.60 16.10 22.60 22.60 18.10 18.70 26.50 12.90 22.60
2023-09-01    8.50 18.00 18.50 20.90 18.00 17.10 20.40 20.40 15.60 19.00
2023-10-01    8.90 19.40 18.30 23.00 25.10 14.10 20.90 22.50 19.40 18.30
2023-11-01   12.50 18.40 16.40 15.80 19.70 13.80 22.40 17.10 16.40 15.80
2023-12-01   12.40 15.60 19.90 18.30 21.00 23.70 19.40 20.40 20.40 18.30
2024-01-01    9.20 16.80 18.90 14.10 18.90 19.50 17.30 19.50 20.00 16.20
2024-02-01    9.30 19.20 16.50 18.70 20.30 12.10 21.40 19.80 23.10 19.20
2024-03-01   12.60 18.00 18.00 18.60 17.50 21.90 22.40 23.50 18.60 19.10
2024-04-01   10.10 21.30 20.20 16.90 20.80 24.70 20.20 18.50 18.50   NaN
2024-05-01   11.30 11.80 23.10 22.60 19.40 19.40 16.10 17.70   NaN   NaN
2024-06-01   10.70 19.70 26.40 17.40 20.20 16.30 16.90   NaN   NaN   NaN
2024-07-01    3.40 10.30  6.90   NaN 17.20   NaN   NaN   NaN   NaN   NaN
2024-08-01    2.90  2.90  2.90  2.90 11.40   NaN   NaN   NaN   NaN   NaN
2024-09-01    5.00   NaN 15.00 10.00   NaN   NaN   NaN   NaN   NaN   NaN
2024-10-01    3.40  3.40   NaN   NaN   NaN   NaN   NaN   NaN   NaN   NaN
2024-11-01    5.70 14.30   NaN   NaN   NaN   NaN   NaN   NaN   NaN   NaN
2024-12-01     NaN   NaN   NaN   NaN   NaN   NaN   NaN   NaN   NaN   NaN

cohort_index    10    11
cohort_month
2023-01-01   27.40 30.00
2023-02-01   29.80 24.10
2023-03-01   25.40 27.90
2023-04-01   28.00 26.10
2023-05-01   30.70 29.40
2023-06-01   25.40 28.50
2023-07-01   19.70 20.80
2023-08-01   18.10 16.10
2023-09-01   21.80 18.50
2023-10-01   22.00 18.30
2023-11-01   22.40 19.70
2023-12-01   21.00 23.10
2024-01-01   18.90 17.80
2024-02-01   14.30   NaN
2024-03-01     NaN   NaN
2024-04-01     NaN   NaN
2024-05-01     NaN   NaN
2024-06-01     NaN   NaN
2024-07-01     NaN   NaN
2024-08-01     NaN   NaN
2024-09-01     NaN   NaN
2024-10-01     NaN   NaN
2024-11-01     NaN   NaN
2024-12-01     NaN   NaN
```

Calculating Monthly Active Users (MAU)

In [68]:

```python
# Extract month from each session date
sessions["activity_month"] = (
    sessions["session_date"].dt.to_period("M")
)

# Count unique active customers per month
```

```
mau = (
    sessions.groupby("activity_month")["customer_id"]
    .nunique()
    .reset_index(name="MAU")
)

print(mau.to_string(index=False))
print("\nAverage MAU:", round(mau["MAU"].mean()))
print("Highest MAU:", mau["MAU"].max())
print("Lowest MAU:", mau["MAU"].min())
```

```
activity_month  MAU
       2023-01 2720
       2023-02 2561
       2023-03 2763
       2023-04 2640
       2023-05 2758
       2023-06 2685
       2023-07 2736
       2023-08 2725
       2023-09 2635
       2023-10 2752
       2023-11 2707
       2023-12 2825
       2024-01 2664
       2024-02 2633
       2024-03 2808
       2024-04 2649
       2024-05 2783
       2024-06 2676
       2024-07 2748
       2024-08 2789
       2024-09 2728
       2024-10 2729
       2024-11 2645
       2024-12 2741

Average MAU: 2712
Highest MAU: 2825
Lowest MAU: 2561
```

Weekly Active Users (WAU)

In [69]:

```
sessions["activity_week"] = (
    sessions["session_date"].dt.to_period("W-SUN")
)

wau = (
    sessions.groupby("activity_week")["customer_id"]
    .nunique()
    .reset_index(name="WAU")
)

print(wau.head(10).to_string(index=False))
print("\nAverage WAU:", round(wau["WAU"].mean()))
print("Highest WAU:", wau["WAU"].max())
print("Lowest WAU:", wau["WAU"].min())
```

```
        activity_week  WAU
2022-12-26/2023-01-01  118
2023-01-02/2023-01-08  711
2023-01-09/2023-01-15  729
2023-01-16/2023-01-22  732
2023-01-23/2023-01-29  697
2023-01-30/2023-02-05  726
2023-02-06/2023-02-12  713
2023-02-13/2023-02-19  767
2023-02-20/2023-02-26  698
2023-02-27/2023-03-05  772

Average WAU: 718
Highest WAU: 793
Lowest WAU: 118
```

DAU (Daily Active Users)

In [70]:
```python
sessions["activity_date"] = sessions["session_date"].dt.date

dau = (
    sessions.groupby("activity_date")["customer_id"]
    .nunique()
    .reset_index(name="DAU")
)

print(dau.head(10).to_string(index=False))

print("\nAverage DAU:", round(dau["DAU"].mean()))
print("Highest DAU:", dau["DAU"].max())
print("Lowest DAU:", dau["DAU"].min())
```

```
activity_date  DAU
   2023-01-01  118
   2023-01-02  110
   2023-01-03   95
   2023-01-04  110
   2023-01-05  108
   2023-01-06  111
   2023-01-07  106
   2023-01-08   99
   2023-01-09  100
   2023-01-10  111

Average DAU: 109
Highest DAU: 146
Lowest DAU: 80
```

DAU/MAU Stickiness

In [71]:
```python
# Average DAU for each month
daily_monthly = dau.copy()

daily_monthly["activity_date"] = pd.to_datetime(
    daily_monthly["activity_date"]
)

daily_monthly["activity_month"] = (
    daily_monthly["activity_date"].dt.to_period("M")
)
```

```python
avg_dau_monthly = (
    daily_monthly.groupby("activity_month")["DAU"]
    .mean()
    .reset_index(name="Average_DAU")
)

# Merge with monthly active users
mau_stickiness = mau.copy()

mau_stickiness["activity_month"] = (
    mau_stickiness["activity_month"].astype("period[M]")
)

stickiness = avg_dau_monthly.merge(
    mau_stickiness,
    on="activity_month",
    how="inner"
)

stickiness["Stickiness_pct"] = (
    stickiness["Average_DAU"] / stickiness["MAU"] * 100
)

print(stickiness.round(2).to_string(index=False))

print(
    "\nAverage Stickiness:",
    round(stickiness["Stickiness_pct"].mean(), 2),
    "%"
)
```

```
activity_month  Average_DAU  MAU  Stickiness_pct
       2023-01       107.23 2720            3.94
       2023-02       110.21 2561            4.30
       2023-03       110.16 2763            3.99
       2023-04       107.93 2640            4.09
       2023-05       108.97 2758            3.95
       2023-06       108.07 2685            4.02
       2023-07       107.26 2736            3.92
       2023-08       106.97 2725            3.93
       2023-09       107.13 2635            4.07
       2023-10       107.87 2752            3.92
       2023-11       109.37 2707            4.04
       2023-12       112.23 2825            3.97
       2024-01       105.10 2664            3.95
       2024-02       109.69 2633            4.17
       2024-03       111.81 2808            3.98
       2024-04       107.10 2649            4.04
       2024-05       110.00 2783            3.95
       2024-06       108.63 2676            4.06
       2024-07       109.58 2748            3.99
       2024-08       110.94 2789            3.98
       2024-09       110.27 2728            4.04
       2024-10       106.87 2729            3.92
       2024-11       106.03 2645            4.01
       2024-12       112.73 2741            4.11

Average Stickiness: 4.01 %
```

Conversion Funnel Analysis

In [72]:
```
total_sessions = sessions["session_id"].nunique()

converted_sessions = sessions.loc[
    sessions["converted"].eq(1), "session_id"
].nunique()

sessions_with_cart = sessions.loc[
    sessions["cart_additions"] > 0, "session_id"
].nunique()

conversion_rate = converted_sessions / total_sessions * 100
cart_rate = sessions_with_cart / total_sessions * 100

print("Total Sessions:", total_sessions)
print("Converted Sessions:", converted_sessions)
print("Sessions with Cart Additions:", sessions_with_cart)
print("Session Conversion Rate:", round(conversion_rate, 2), "%")
print("Sessions with Cart Additions Rate:", round(cart_rate, 2), "%")
```

```
Total Sessions: 80000
Converted Sessions: 8648
Sessions with Cart Additions: 37229
Session Conversion Rate: 10.81 %
Sessions with Cart Additions Rate: 46.54 %
```

Analyzing the Funnel

In [73]:
```
cart_sessions = sessions[sessions["cart_additions"] > 0]

cart_converted = cart_sessions[
    cart_sessions["converted"] == 1
]

print("Sessions with cart additions:", len(cart_sessions))
print("Converted sessions with cart additions:", len(cart_converted))

print(
    "Cart-session conversion rate:",
    round(len(cart_converted) / len(cart_sessions) * 100, 2),
    "%"
)

print(
    "Converted sessions without cart additions:",
    len(sessions[
        (sessions["cart_additions"] == 0) &
        (sessions["converted"] == 1)
    ])
)
```

```
Sessions with cart additions: 37229
Converted sessions with cart additions: 8648
Cart-session conversion rate: 23.23 %
Converted sessions without cart additions: 0
```

Comparing Conversion Rates by Channel

In [74]:
```
channel_analysis = (
    sessions.groupby("channel")
    .agg(
        total_sessions=("session_id", "nunique"),
```

```
        converted_sessions=("converted", "sum"),
        sessions_with_cart=("cart_additions", lambda x: (x > 0).sum())
    )
    .reset_index()
)

channel_analysis["conversion_rate_pct"] = (
    channel_analysis["converted_sessions"]
    / channel_analysis["total_sessions"] * 100
)

channel_analysis["cart_rate_pct"] = (
    channel_analysis["sessions_with_cart"]
    / channel_analysis["total_sessions"] * 100
)

channel_analysis = channel_analysis.sort_values(
    "conversion_rate_pct", ascending=False
)

print(channel_analysis.round(2).to_string(index=False))
```

```
   channel  total_sessions  converted_sessions  sessions_with_cart  conversion_rate_pct  cart
_rate_pct
    social           12071                1361                5644                11.27
46.76
  referral            8119                 911                3823                11.22
47.09
paid_search           15915                1741                7365                10.94
46.28
    direct           12067                1295                5595                10.73
46.37
     email           11854                1256                5573                10.60
47.01
   organic           19974                2084                9229                10.43
46.21
```

Conversion Rate by Device

In [75]:

```
device_analysis = (
    sessions.groupby("device")
    .agg(
        total_sessions=("session_id", "nunique"),
        converted_sessions=("converted", "sum")
    )
    .reset_index()
)

device_analysis["conversion_rate_pct"] = (
    device_analysis["converted_sessions"]
    / device_analysis["total_sessions"] * 100
)

device_analysis = device_analysis.sort_values(
    "conversion_rate_pct", ascending=False
)

print(device_analysis.round(2).to_string(index=False))
```

device	total_sessions	converted_sessions	conversion_rate_pct
desktop	28053	3106	11.07
mobile	40010	4300	10.75
tablet	11937	1242	10.40

Conversion Rate by Customer Segment

In [76]:
```
session_segments = sessions.merge(
    customers[["customer_id", "segment"]],
    on="customer_id",
    how="left",
    validate="many_to_one"
)

segment_analysis = (
    session_segments.groupby("segment")
    .agg(
        total_sessions=("session_id", "nunique"),
        converted_sessions=("converted", "sum")
    )
    .reset_index()
)

segment_analysis["conversion_rate_pct"] = (
    segment_analysis["converted_sessions"]
    / segment_analysis["total_sessions"] * 100
)

segment_analysis = segment_analysis.sort_values(
    "conversion_rate_pct", ascending=False
)

print(segment_analysis.round(2).to_string(index=False))
```

segment	total_sessions	converted_sessions	conversion_rate_pct
VIP	18399	3371	18.32
Premium	22928	2699	11.77
Regular	26525	2028	7.65
Budget Shopper	11394	536	4.70
Occasional Visitor	754	14	1.86

Funnel Analysis by Session Depth

In [77]:
```
sessions["page_group"] = pd.cut(
    sessions["pages_viewed"],
    bins=[0, 2, 5, 10, float("inf")],
    labels=["1-2 pages", "3-5 pages", "6-10 pages", "10+ pages"]
)

page_analysis = (
    sessions.groupby("page_group", observed=True)
    .agg(
        total_sessions=("session_id", "nunique"),
        converted_sessions=("converted", "sum")
    )
    .reset_index()
)

page_analysis["conversion_rate_pct"] = (
    page_analysis["converted_sessions"]
    / page_analysis["total_sessions"] * 100
```

```
)

print(page_analysis.round(2).to_string(index=False))
```

```
page_group  total_sessions  converted_sessions  conversion_rate_pct
 1-2 pages           33366                3199                 9.59
 3-5 pages           17478                1775                10.16
6-10 pages           14302                1625                11.36
 10+ pages           14854                2049                13.79
```

Conversion Rate by Traffic Channel and Device

In [78]:
```
channel_device = (
    sessions.groupby(["channel", "device"])
    .agg(
        total_sessions=("session_id", "nunique"),
        converted_sessions=("converted", "sum")
    )
    .reset_index()
)

channel_device["conversion_rate_pct"] = (
    channel_device["converted_sessions"]
    / channel_device["total_sessions"] * 100
)

channel_device = channel_device.sort_values(
    "conversion_rate_pct", ascending=False
)

print(channel_device.round(2).to_string(index=False))
```

```
   channel  device  total_sessions  converted_sessions  conversion_rate_pct
    social desktop            4189                 483                11.53
  referral desktop            2851                 324                11.36
paid_search desktop            5555                 627                11.29
    social  mobile            6059                 682                11.26
    direct desktop            4309                 482                11.19
  referral  mobile            4052                 453                11.18
  referral  tablet            1216                 134                11.02
paid_search  mobile            7961                 860                10.80
    social  tablet            1823                 196                10.75
     email desktop            4103                 440                10.72
   organic desktop            7046                 750                10.64
     email  mobile            5969                 634                10.62
    direct  tablet            1770                 188                10.62
paid_search  tablet            2399                 254                10.59
   organic  mobile            9981                1046                10.48
    direct  mobile            5988                 625                10.44
     email  tablet            1782                 182                10.21
   organic  tablet            2947                 288                 9.77
```

Conversion Rate Optimization

In [79]:
```
bounce_analysis = (
    sessions.groupby("bounced")
    .agg(
        total_sessions=("session_id", "nunique"),
        converted_sessions=("converted", "sum")
    )
    .reset_index()
```

```
)

bounce_analysis["conversion_rate_pct"] = (
    bounce_analysis["converted_sessions"]
    / bounce_analysis["total_sessions"] * 100
)

print(bounce_analysis.round(2).to_string(index=False))
```

```
bounced  total_sessions  converted_sessions  conversion_rate_pct
      0           72746                8000                11.00
      1            7254                 648                 8.93
```

Lifetime Value Regression

In [80]:
```
print(customers["lifetime_value"].describe().round(2))
print("\nMissing values:", customers["lifetime_value"].isna().sum())
```

```
count   10,000.00
mean     1,544.61
std      1,426.69
min         71.21
25%        587.94
50%      1,043.80
75%      1,952.15
max      9,353.25
Name: lifetime_value, dtype: float64

Missing values: 0
```

Preparing Features for LTV Regression

In [81]:
```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import OneHotEncoder
from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score

# Select features
features = [
    "age", "gender", "country", "segment",
    "email_opt_in", "has_app", "is_churned"
]

X = customers[features].copy()
y = customers["lifetime_value"].copy()

# Identify feature types
categorical_features = ["gender", "country", "segment"]
numeric_features = [
    "age", "email_opt_in", "has_app", "is_churned"
]

# Preprocessing
preprocessor = ColumnTransformer([
    ("cat", OneHotEncoder(handle_unknown="ignore"), categorical_features),
    ("num", SimpleImputer(strategy="median"), numeric_features)
])
```

```
# Model pipeline
model = Pipeline([
    ("preprocessor", preprocessor),
    ("regressor", RandomForestRegressor(
        n_estimators=100,
        random_state=42,
        n_jobs=-1
    ))
])

# Split data
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

print("Training rows:", len(X_train))
print("Testing rows:", len(X_test))
```

```
Training rows: 8000
Testing rows: 2000
```

Train the LTV Regression Model

In [82]:

```
# Train model
model.fit(X_train, y_train)

# Predict LTV on unseen test data
y_pred = model.predict(X_test)

# Evaluate model
mae = mean_absolute_error(y_test, y_pred)
rmse = mean_squared_error(y_test, y_pred) ** 0.5
r2 = r2_score(y_test, y_pred)

print("MAE:", round(mae, 2))
print("RMSE:", round(rmse, 2))
print("R² Score:", round(r2, 4))
```

```
MAE: 329.97
RMSE: 538.3
R² Score: 0.8484
```

Identifying Important Features

In [83]:

```
# Getting feature names after preprocessing
feature_names = model.named_steps["preprocessor"].get_feature_names_out()

# Getting Random Forest feature importance
importance = model.named_steps["regressor"].feature_importances_

feature_importance = (
    pd.DataFrame({
        "feature": feature_names,
        "importance": importance
    })
    .sort_values("importance", ascending=False)
)

print(feature_importance.head(10).round(4).to_string(index=False))
```

```
                    feature  importance
          cat__segment_VIP        0.67
      cat__segment_Premium        0.19
                  num__age        0.05
      cat__segment_Regular        0.02
         num__email_opt_in        0.01
cat__segment_Budget Shopper        0.01
             num__has_app        0.01
            cat__gender_M        0.01
            cat__gender_F        0.00
          cat__country_US        0.00
```

Churn Prediction

In [84]:
```python
print(customers["is_churned"].value_counts())
print("\nChurn percentage:")
print(
    (customers["is_churned"].value_counts(normalize=True) * 100).round(2)
)
```

```
is_churned
0    8306
1    1694
Name: count, dtype: int64

Churn percentage:
is_churned
0   83.06
1   16.94
Name: proportion, dtype: float64
```

Preparing the Churn Prediction Model

In [85]:
```python
from sklearn.model_selection import train_test_split
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import OneHotEncoder
from sklearn.impute import SimpleImputer
from sklearn.pipeline import Pipeline
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import classification_report, roc_auc_score

# Features and target
features = [
    "age", "gender", "country", "segment",
    "email_opt_in", "has_app"
]

X = customers[features].copy()
y = customers["is_churned"]

categorical_features = ["gender", "country", "segment"]
numeric_features = ["age", "email_opt_in", "has_app"]

preprocessor = ColumnTransformer([
    ("cat", OneHotEncoder(handle_unknown="ignore"), categorical_features),
    ("num", SimpleImputer(strategy="median"), numeric_features)
])

churn_model = Pipeline([
    ("preprocessor", preprocessor),
    ("classifier", RandomForestClassifier(
```

```
        n_estimators=100,
        random_state=42,
        class_weight="balanced",
        n_jobs=-1
    ))
])

X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    random_state=42,
    stratify=y
)

print("Training rows:", len(X_train))
print("Testing rows:", len(X_test))
```

```
Training rows: 8000
Testing rows: 2000
```

Trainning and Evaluating the Churn Model

In [86]:
```
from sklearn.metrics import (
    classification_report,
    roc_auc_score,
    confusion_matrix
)

# Train the model
churn_model.fit(X_train, y_train)

# Predictions
y_pred = churn_model.predict(X_test)
y_prob = churn_model.predict_proba(X_test)[:, 1]

# Evaluation
print("Classification Report:\n")
print(classification_report(y_test, y_pred, zero_division=0))

print("ROC-AUC:", round(roc_auc_score(y_test, y_prob), 4))

print("\nConfusion Matrix:")
print(confusion_matrix(y_test, y_pred))
```

```
Classification Report:

              precision    recall  f1-score   support

           0       0.84      0.83      0.84      1661
           1       0.23      0.25      0.24       339

    accuracy                           0.73      2000
   macro avg       0.54      0.54      0.54      2000
weighted avg       0.74      0.73      0.74      2000

ROC-AUC: 0.5801

Confusion Matrix:
[[1379  282]
 [ 255   84]]
```

Checking Feature Importance

In [87]:
```
feature_names = (
    churn_model.named_steps["preprocessor"]
    .get_feature_names_out()
)

importance = (
    churn_model.named_steps["classifier"]
    .feature_importances_
)

churn_importance = (
    pd.DataFrame({
        "feature": feature_names,
        "importance": importance
    })
    .sort_values("importance", ascending=False)
)

print(churn_importance.head(10).round(4).to_string(index=False))
```

```
                      feature  importance
                     num__age        0.59
            num__email_opt_in        0.04
             cat__segment_VIP        0.04
                 num__has_app        0.04
   cat__segment_Budget Shopper        0.04
cat__segment_Occasional Visitor        0.03
          cat__segment_Premium        0.03
               cat__country_US        0.02
                 cat__gender_F        0.02
          cat__segment_Regular        0.02
```

Recommendation & Market Basket Analysis

In [88]:
```
# Using completed transactions only
basket_data = transactions[
    transactions["status"] == "completed"
][["transaction_id", "product_id"]].drop_duplicates()

print("Completed transaction rows:", len(basket_data))
print("Unique transactions:", basket_data["transaction_id"].nunique())
print("Unique products:", basket_data["product_id"].nunique())
```

```
Completed transaction rows: 68700
Unique transactions: 68700
Unique products: 1000
```

Check Products per Transaction

In [89]:
```
products_per_transaction = (
    basket_data.groupby("transaction_id")["product_id"]
    .nunique()
)

print("Transactions with 1 product:", (products_per_transaction == 1).sum())
print("Transactions with multiple products:", (products_per_transaction > 1).sum())
print("\nProducts per transaction:")
print(products_per_transaction.value_counts().sort_index())
```

```
Transactions with 1 product: 68700
Transactions with multiple products: 0

Products per transaction:
product_id
1    68700
Name: count, dtype: int64
```

In [90]:
```python
completed = transactions[
    transactions["status"] == "completed"
].copy()

customer_product_counts = (
    completed.groupby("customer_id")["product_id"]
    .nunique()
    .sort_values(ascending=False)
)

print("Customers with completed purchases:", customer_product_counts.size)
print("\nUnique products per customer:")
print(customer_product_counts.describe().round(2))

print("\nTop 10 customers by unique products:")
print(customer_product_counts.head(10))
```

```
Customers with completed purchases: 9730

Unique products per customer:
count   9,730.00
mean        7.03
std         4.65
min         1.00
25%         4.00
50%         6.00
75%         9.00
max        29.00
Name: product_id, dtype: float64

Top 10 customers by unique products:
customer_id
C09276    29
C02300    29
C00857    28
C03428    28
C01335    27
C05846    27
C07591    27
C08856    27
C04973    26
C08350    26
Name: product_id, dtype: int64
```

Top 10 Most Purchased Products

In [91]:
```python
# Keeping completed transactions
completed = transactions[
    transactions["status"] == "completed"
].copy()

# Count purchases for each product
top_products = (
    completed.groupby("product_id")
```

```
    .agg(
        purchase_count=("transaction_id", "nunique"),
        total_quantity=("quantity", "sum")
    )
    .reset_index()
    .merge(
        products[["product_id", "product_name", "category", "price"]],
        on="product_id",
        how="left"
    )
    .sort_values("purchase_count", ascending=False)
)

# Display top 10 products
top_products[
    ["product_id", "product_name", "category",
     "purchase_count", "total_quantity", "price"]
].head(10)
```

Out[91]:

	product_id	product_name	category	purchase_count	total_quantity	price
405	P0405	PureBrand Pet Supplies #405	Pet Supplies	103	143	22.16
702	P0702	FitGear Home & Garden #702	Home & Garden	101	148	74.59
955	P0955	PetLife Sports #955	Sports	89	128	71.68
190	P0190	ClassicCo Home & Garden #190	Home & Garden	88	113	58.50
792	P0792	ShineOn Music #792	Music	88	130	10.66
901	P0901	SoundWave Clothing #901	Clothing	88	124	60.80
915	P0915	ClassicCo Jewelry #915	Jewelry	87	116	77.27
790	P0790	EcoLiving Electronics #790	Electronics	87	113	452.95
631	P0631	NovaTech Toys #631	Toys	86	110	53.18
876	P0876	WorkSmart Pet Supplies #876	Pet Supplies	86	131	17.92

1. **Dataset Size:** Analyzed 236,000 rows across five datasets: customers, products, transactions, sessions, and reviews.
2. **Data Quality:** Verified primary IDs, foreign-key relationships, and transaction price consistency; no duplicate rows were found.
3. **Date Inconsistency:** Identified 11,027 transactions and 7,294 sessions occurring before the associated customers' signup dates.
4. **Overall Conversion Rate:** Of 80,000 browsing sessions, 8,648 converted, resulting in a conversion rate of **10.81%**.
5. **Channel Performance:** Social had the highest channel conversion rate at **11.27%**, while organic had the lowest at 10.43%.
6. **Device Performance:** Desktop users had the highest conversion rate at **11.07%**, followed by mobile at 10.75% and tablet at 10.40%.

7. **Customer Segments:** VIP customers had the highest conversion rate at **18.32%**, while Occasional Visitors had the lowest at 1.86%.
8. **Browsing Behavior:** Sessions with 10+ pages viewed had a **13.79% conversion rate**, compared with 9.59% for sessions with 1–2 pages viewed.
9. **Cart Additions:** Approximately **46.54% of sessions** included cart additions, indicating an opportunity to investigate the relationship between cart activity and conversion.
10. **Customer Lifetime Value:** Average customer lifetime value was **1,544.61**, with a median of 1,043.80, indicating variation in customer value.
11. **LTV Prediction:** The Random Forest Regressor achieved an **R^2 score of 0.8484**, with customer segment emerging as the most influential predictor.
12. **Customer Churn:** Approximately **16.94% of customers** were labeled as churned in the dataset.
13. **Churn Model Performance:** The baseline Random Forest Classifier achieved **73% accuracy**, 25% churn recall, and a ROC-AUC of 0.5801, indicating limited churn detection capability.
14. **Product Recommendations:** Product 405 had the highest purchase count among the top 10 products, appearing in **103 completed transactions**.
15. **Market Basket Limitation:** Each transaction contained only one unique product, preventing traditional basket-level association analysis. Popularity-based product recommendations remain feasible.

In [ ]: